Lab : Analyze Network Traffic

Challenge Scenario

You are a network forensic analyst at a medium-sized enterprise. The network team has detected unusual network activities and generated a PCAP file from one of the internal segments during a suspected attack window. Your task is to analyze the PCAP file to identify potential malicious activities, verify if any data was exfiltrated, and assess any command-and-control communications.

Prior Knowledge

Benign Traffic

Benign traffic refers to legitimate network data that does not pose any threat to security. Understanding benign traffic is essential for network analysts because it helps establish a baseline of normal activity, making it easier to spot anomalies or malicious activities.

1. **Web Browsing:** Requests and responses over HTTP/HTTPS that are part of typical user activity.
2. **Email Communications:** SMTP, POP3, and IMAP traffic used for sending and receiving emails.
3. **FTP Transfers:** Normal file transfers using FTP, which might be routine backups or scheduled data transfers.
4. **DNS Queries:** Regular DNS requests that resolve domain names to IP addresses.

Command and Control (C&C) Communications

C&C communications refer to signals and data passed between compromised systems and an attacker's server.

These communications are common in network breaches involving malware.

1. **Control Mechanisms:** C&C can be conducted over various protocols, including IRC, HTTP, HTTPS, or custom protocols.
2. **Purpose:** Allows attackers to remotely manage malware, perform data exfiltration, deploy additional payloads, and more.
3. **Detection Challenges:** C&C traffic is often designed to mimic benign traffic to evade detection.

Steps

Open the PCAP File:

Downloaded the network_traffic.pcap file in my windows VM and Open the provided network_traffic.pcap file in Wireshark or Network Miner.

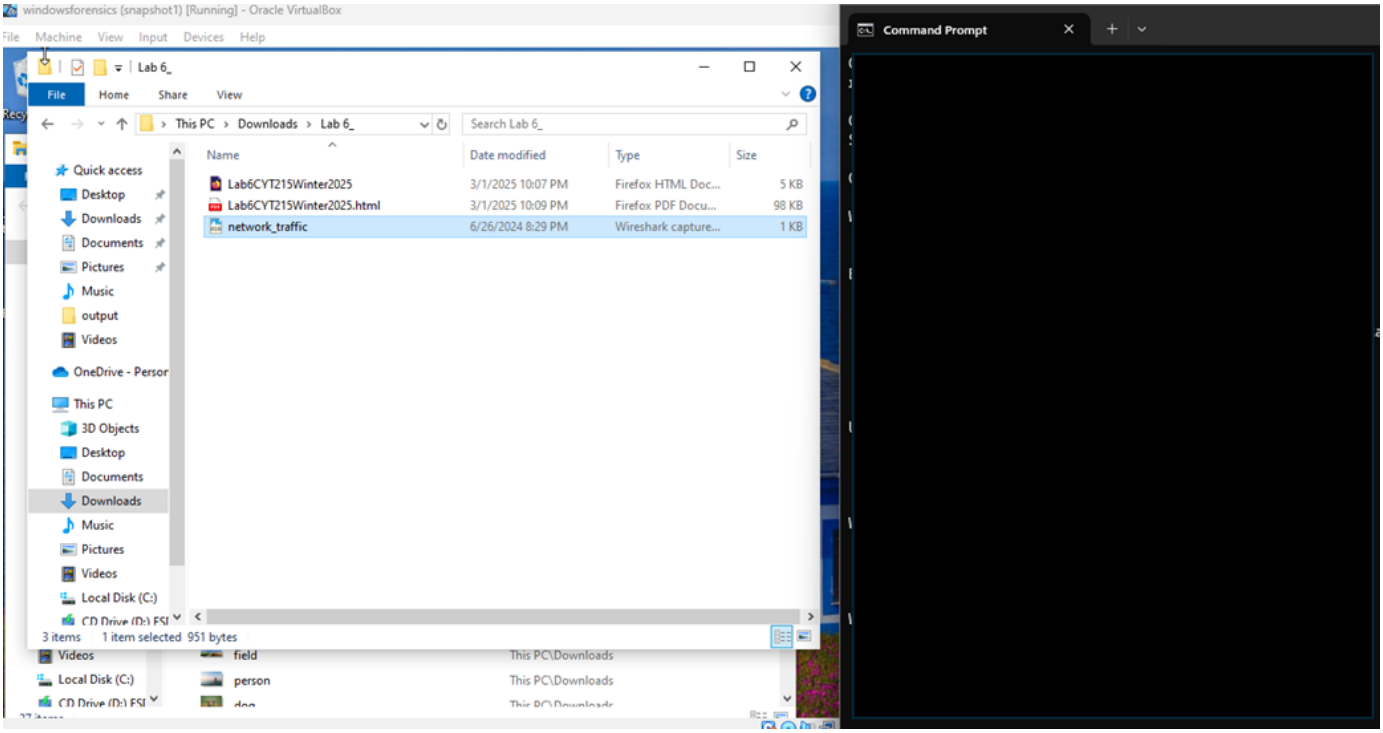


Figure 1:Download the folder in my Windows VM

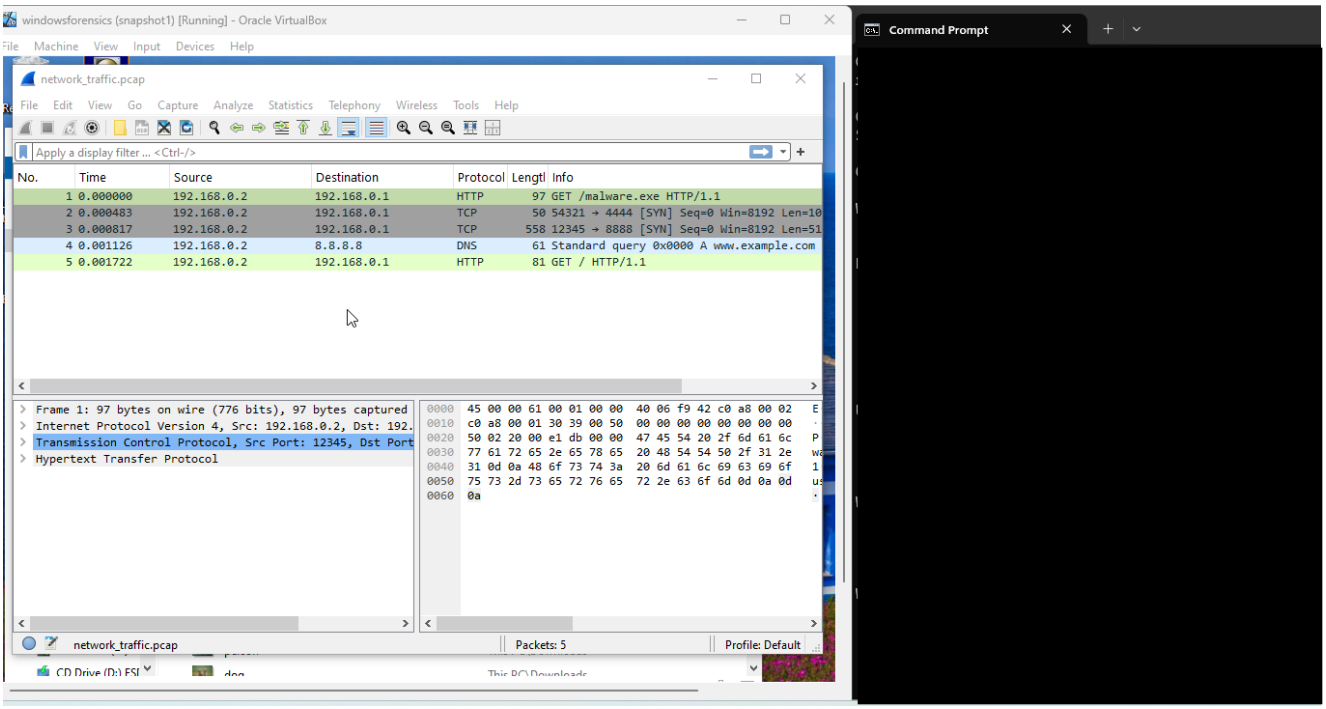


Figure 2:Opened netowrk_traffic.pcap file in Wireshark

Identify Malware Download:

Use the filter: http.request.method == "GET" && http.request.uri contains "malware.exe"

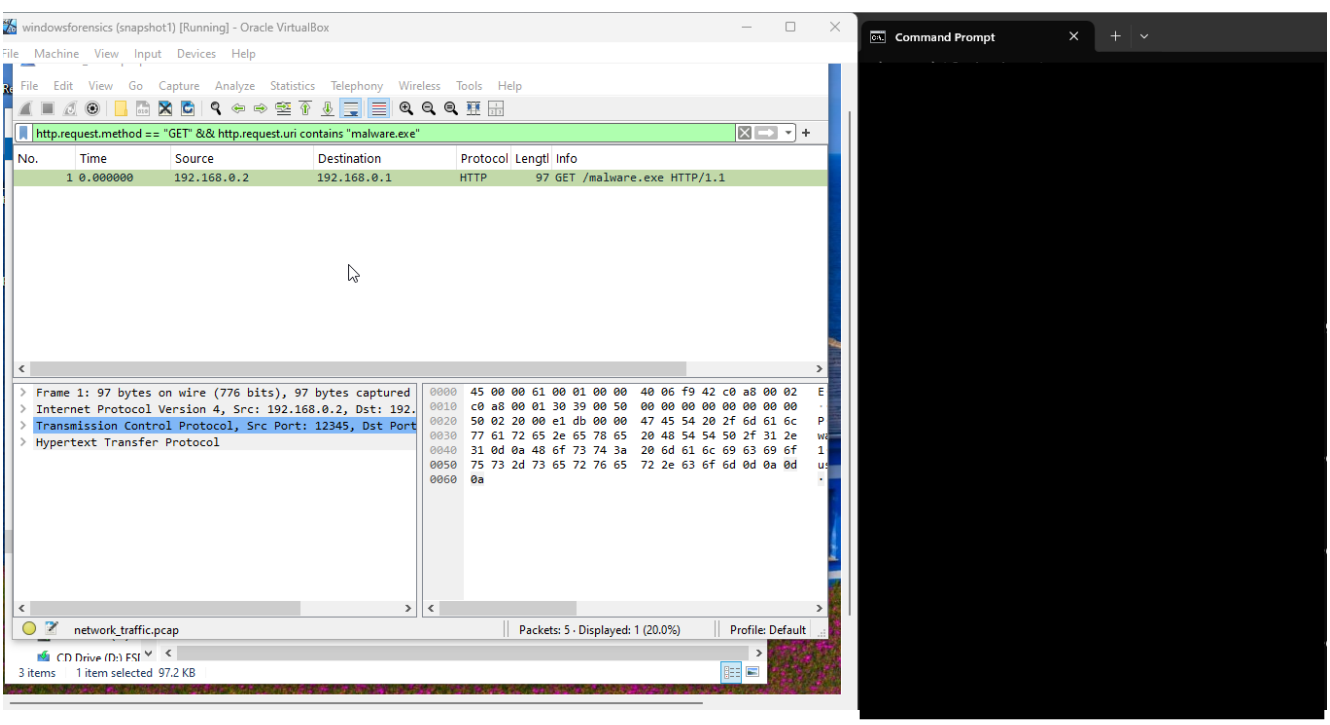


Figure 3:Filtering out to get malware file

Identify the source and destination IP addresses and note the HTTP host header.

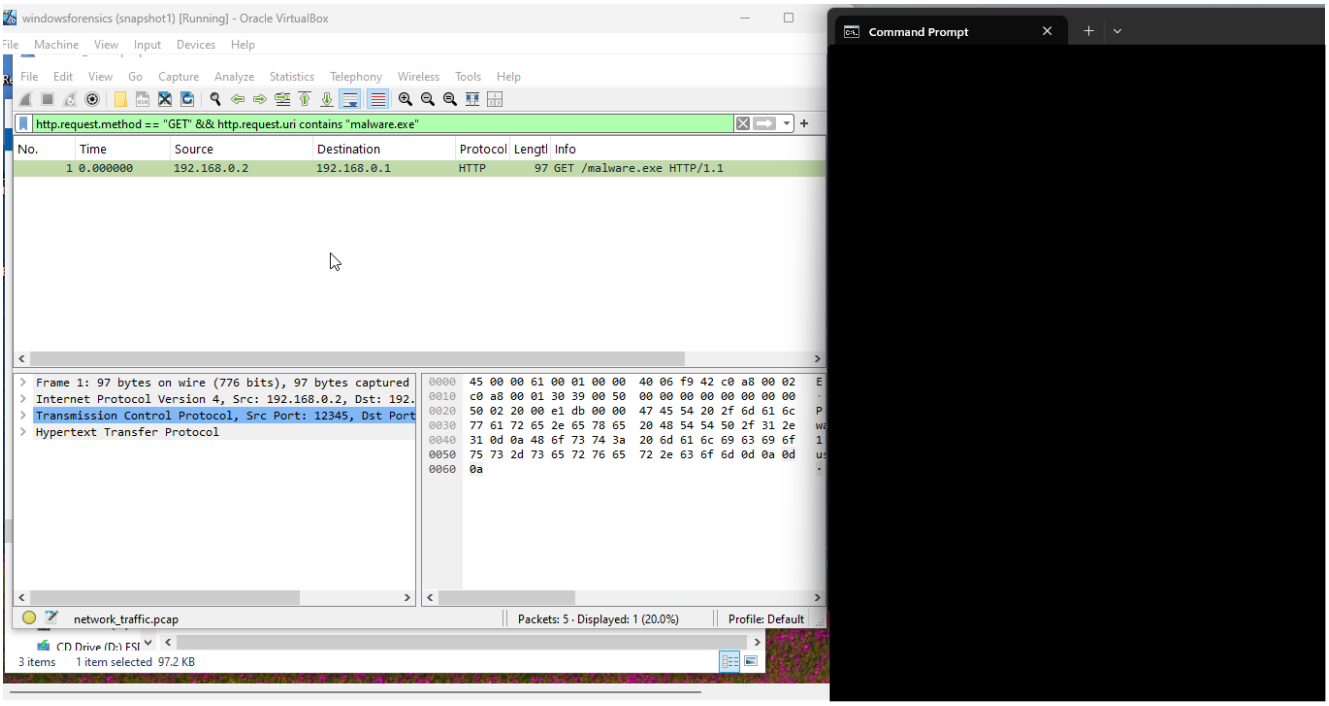


Figure 4:Filtering out to get malware file(2)

From the figure 3 & 4, I can see the source IP address is 192.168.0.2 and the destination IP address is 192.168.0.1.

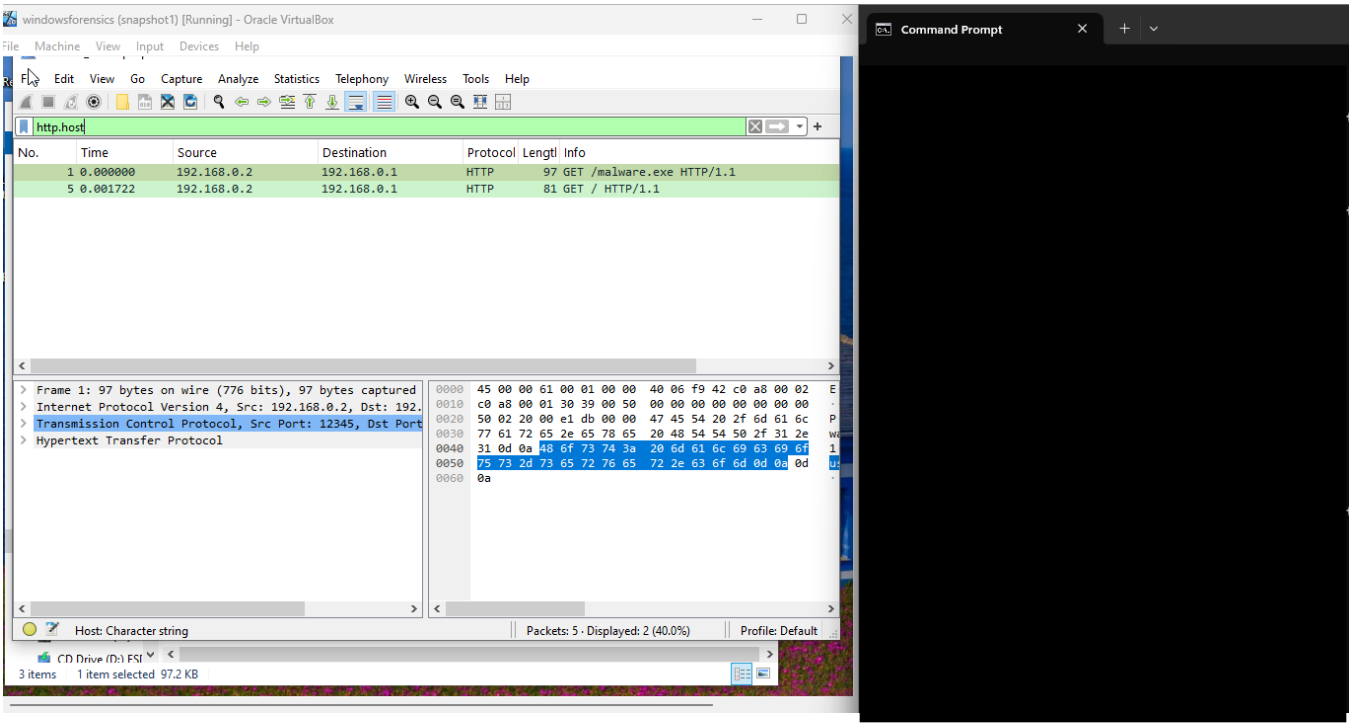


Figure 5:HTTP host header

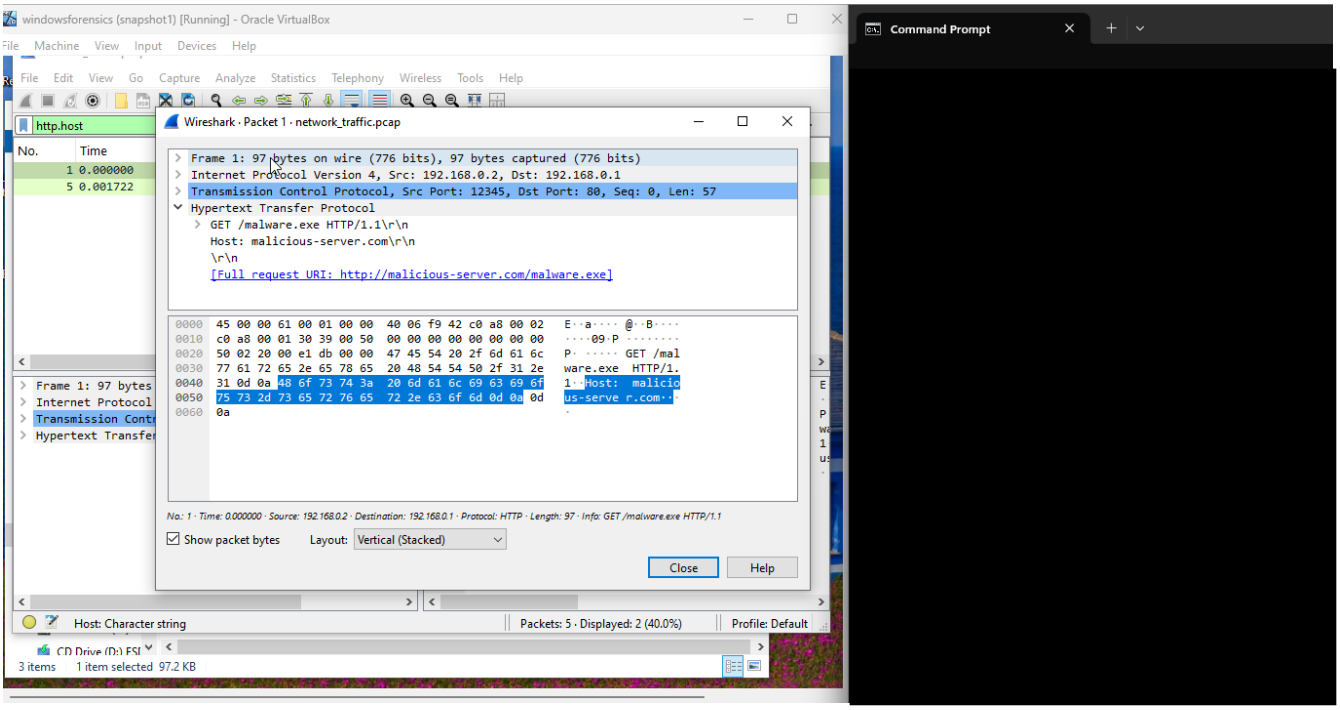


Figure 6:Analyzing HTTP host header

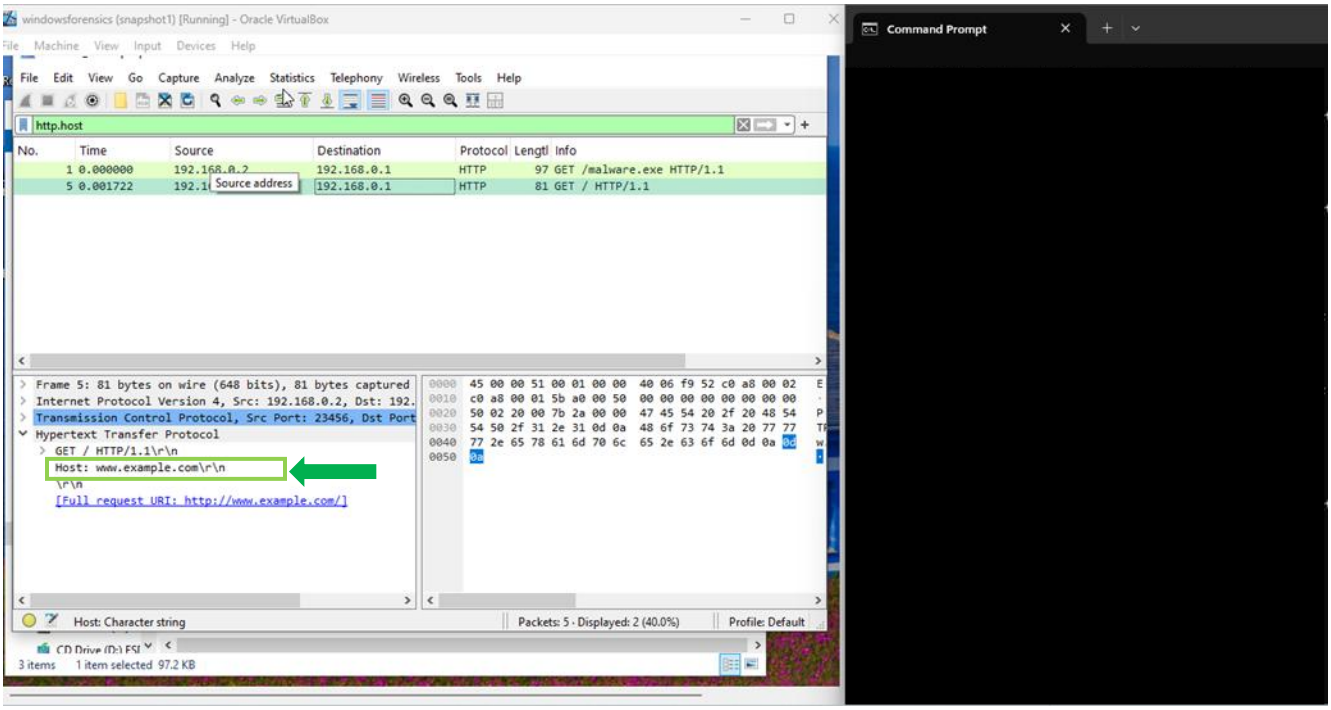


Figure 7:Information about HTTP

Another way to view host header is adding it as column.

Click HTTP host line and select apply as a column. Then I can see HTTP host column.

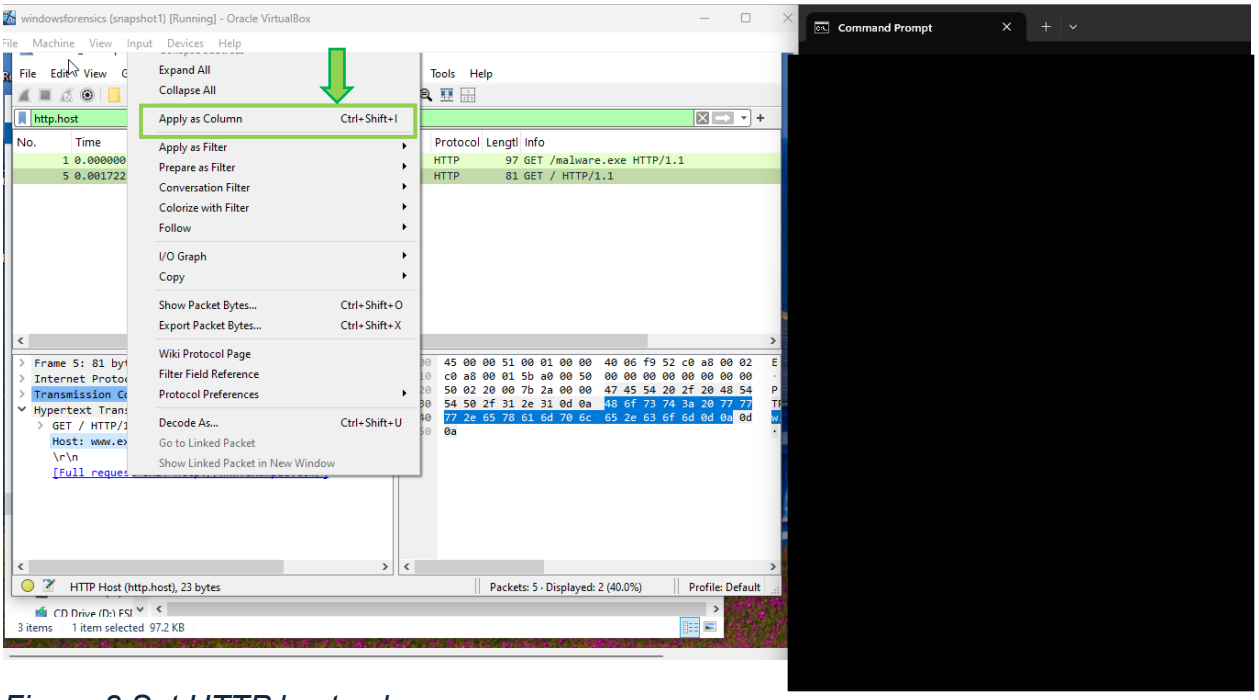
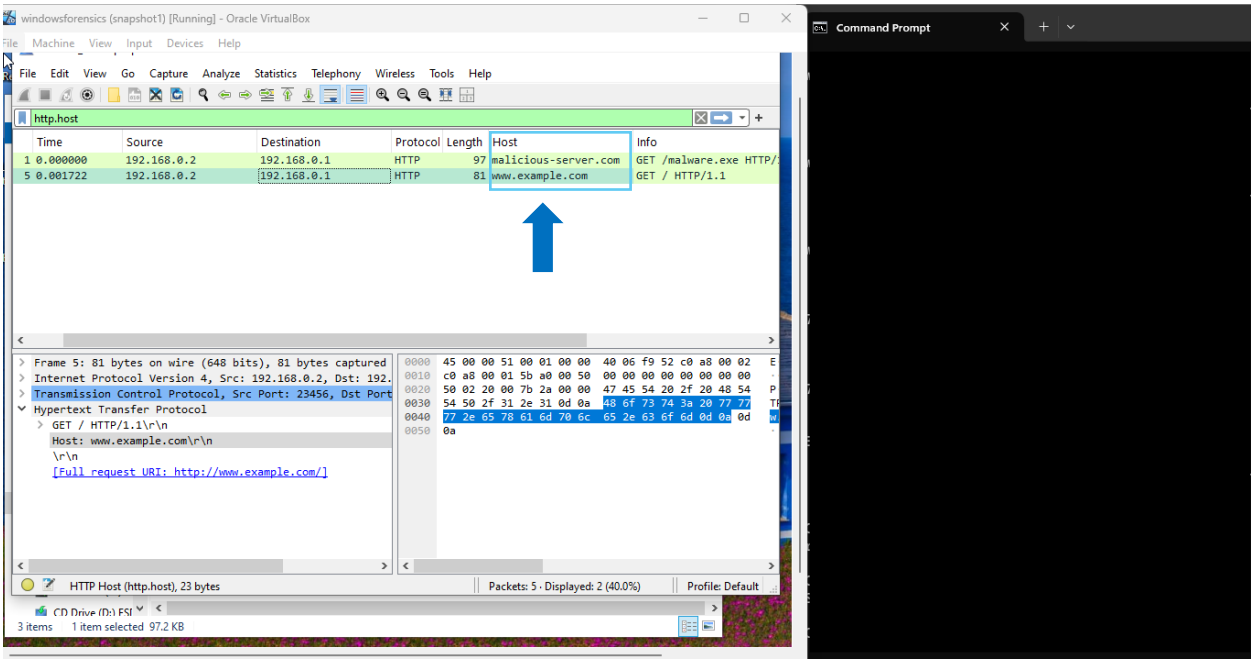


Figure 8:Set HTTP host column



Identify the source and destination IP addresses and note the HTTP host header.

The HTTP host header is example.com.

Discuss the implications of malware being downloaded over HTTP.

Implications of Malware Being Downloaded Over HTTP

When malware is downloaded over **HTTP** instead of **HTTPS**, it introduces several security risks that can lead to severe consequences.

1. Lack of Encryption Enables Man-in-the-Middle (MITM) Attacks

HTTP does not encrypt data, meaning attackers can intercept and modify it.MITM attackers can inject malicious payloads into legitimate downloads.

Example: An attacker positioned on a public Wi-Fi network could replace a software update file with malware.

2. Increased Risk of Malware Injection

Since HTTP does not verify data integrity, attackers can alter files in transit.A user downloading software over HTTP risks receiving a tampered version that contains malware.

Example: If a legitimate website is compromised, its downloads over HTTP can be replaced with trojanized software.

3. Lack of Authenticity Verification

HTTPS ensures certificate validation to verify a website's identity, but HTTP lacks this mechanism. Attackers can impersonate trusted websites using DNS spoofing or phishing attacks.

Example: Attackers make a fake website. That fake website distributes malware-laced downloads.

4. Exploitation via Content Delivery Networks (CDNs)

Some websites host downloads via CDNs that may use HTTP links. Attackers can exploit CDN-hosted files to serve malicious payloads.

Example: A CDN delivering outdated or backdoored versions of a software package.

5. Increased Likelihood of Drive-by Downloads

Malvertising (Malicious Advertising) campaigns often exploit HTTP vulnerabilities. Users visiting HTTP websites can have malware automatically downloaded and executed.

Example: A compromised ad network delivering JavaScript that triggers malware downloads.

6. No Secure Browser Warnings

Modern browsers flag HTTP as insecure, but many users ignore these warnings. Unlike HTTPS, which shows a security indicator, HTTP does not alert users when a download is modified.

Example: A malicious actor distributing cracked software via HTTP download links.

7. Evasion of Network Security Controls

Many network security tools focus on HTTPS decryption for malware detection. HTTP traffic might evade security controls if not monitored effectively.

Example: Legacy security appliances may not inspect unencrypted traffic efficiently.

Investigate Command and Control Communication:

Use the filter: tcp.flags.syn == 1 && tcp.port == 4444

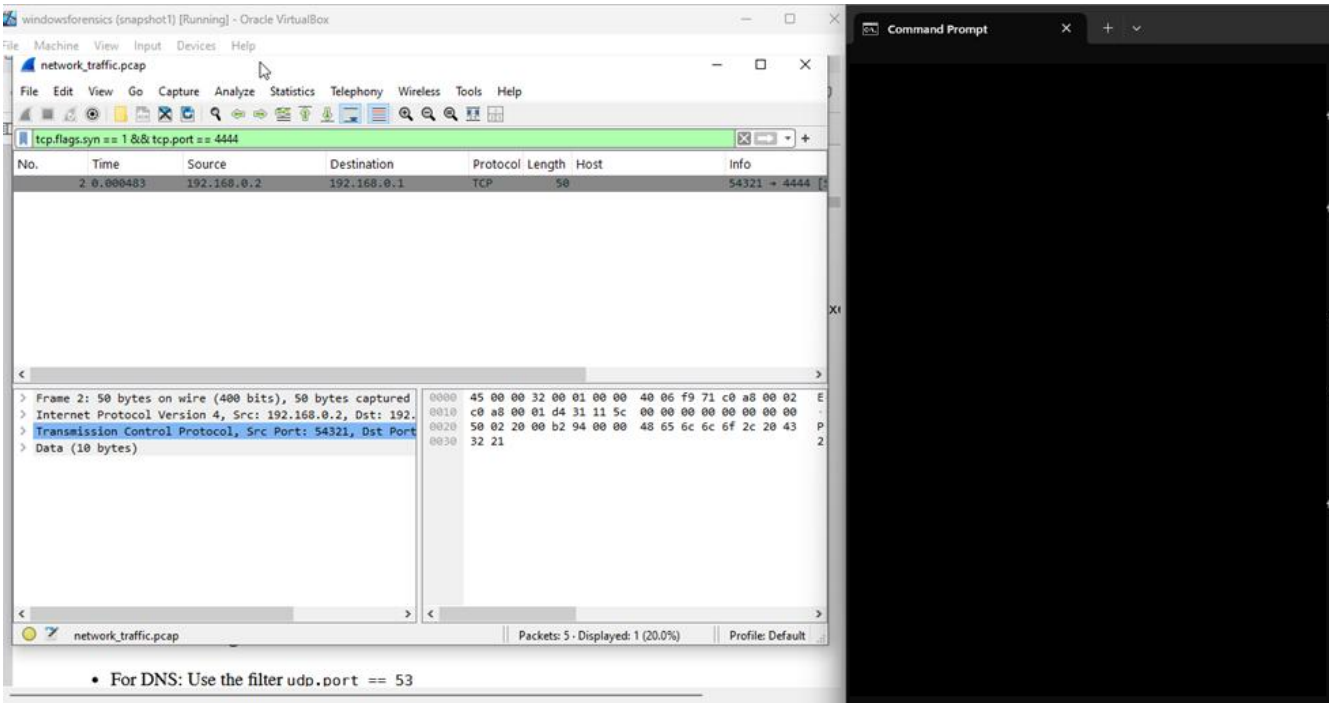


Figure 9:Applied the filter for TCP

Follow the TCP stream to view communication.

Clicked the column that is shown in figure 9&10 and clicked the follow option from there I selected TCP stream to view communication.

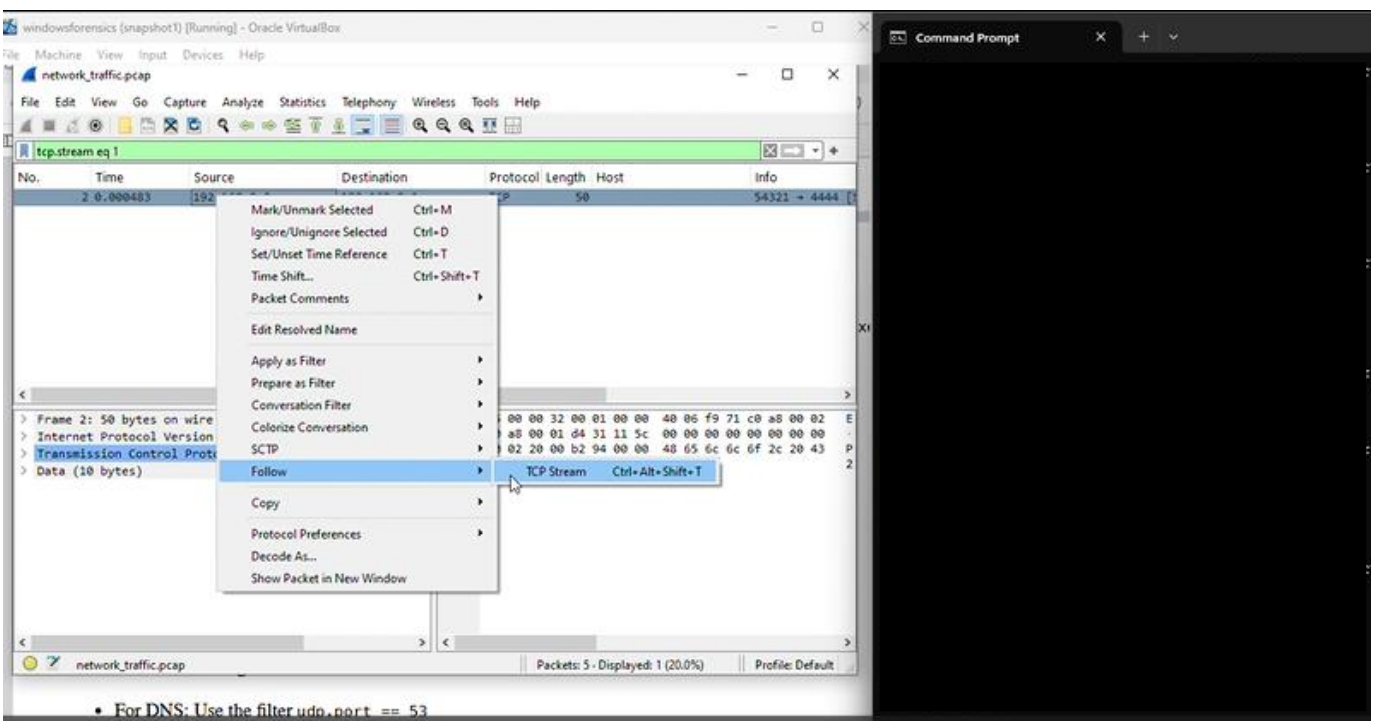


Figure 10:Follow TCP stream

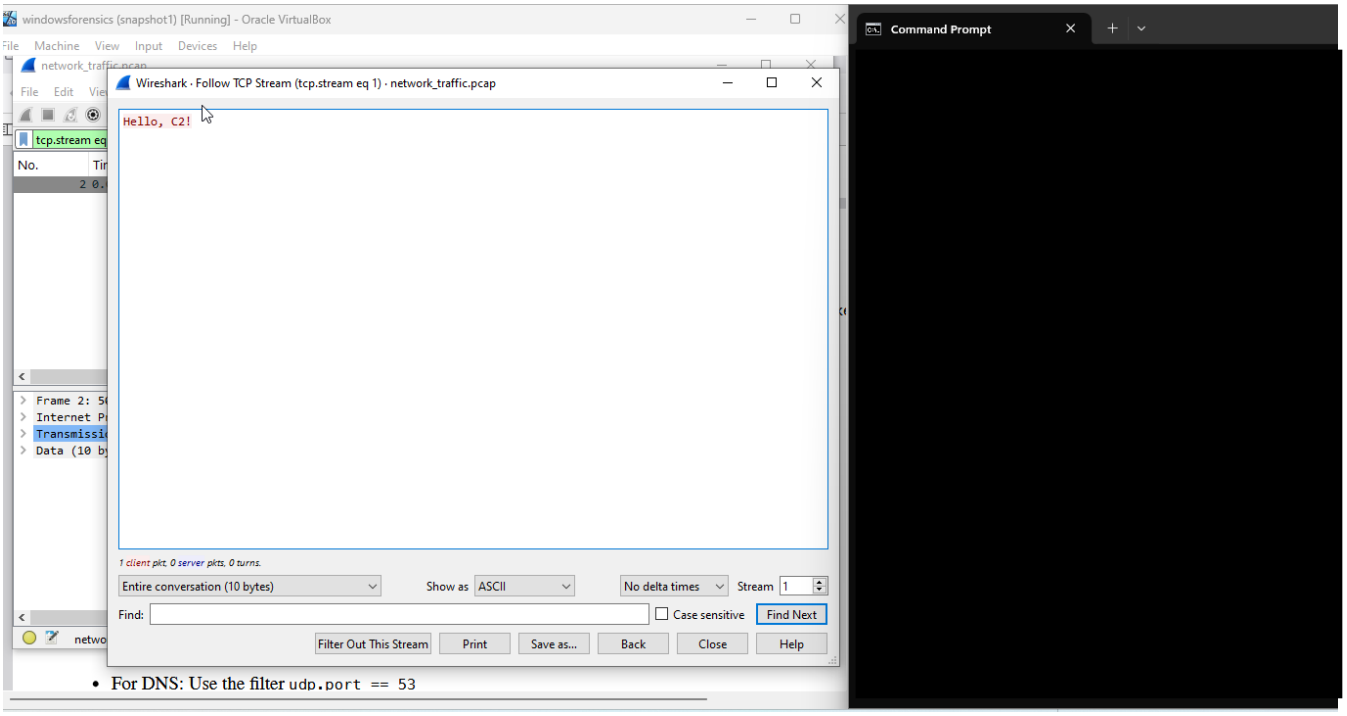


Figure 11:Communication

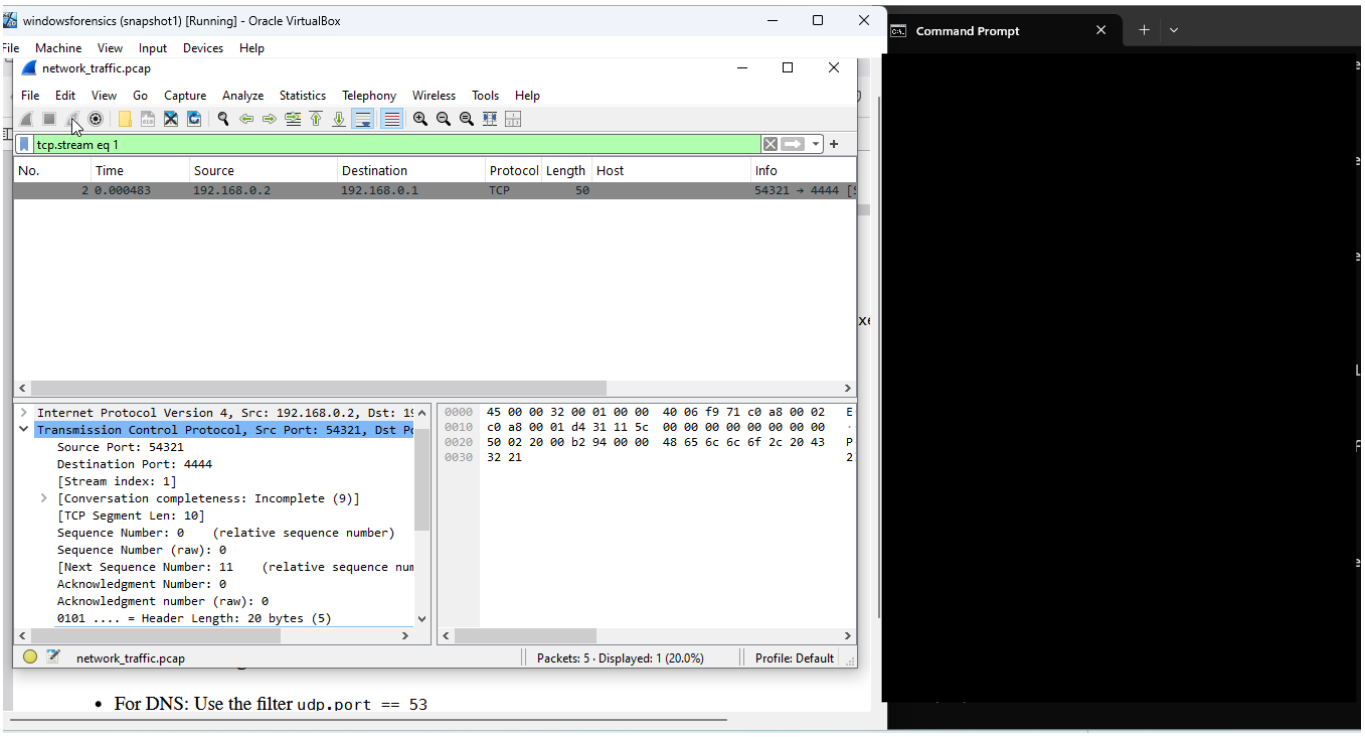


Figure 12:Details about command & control

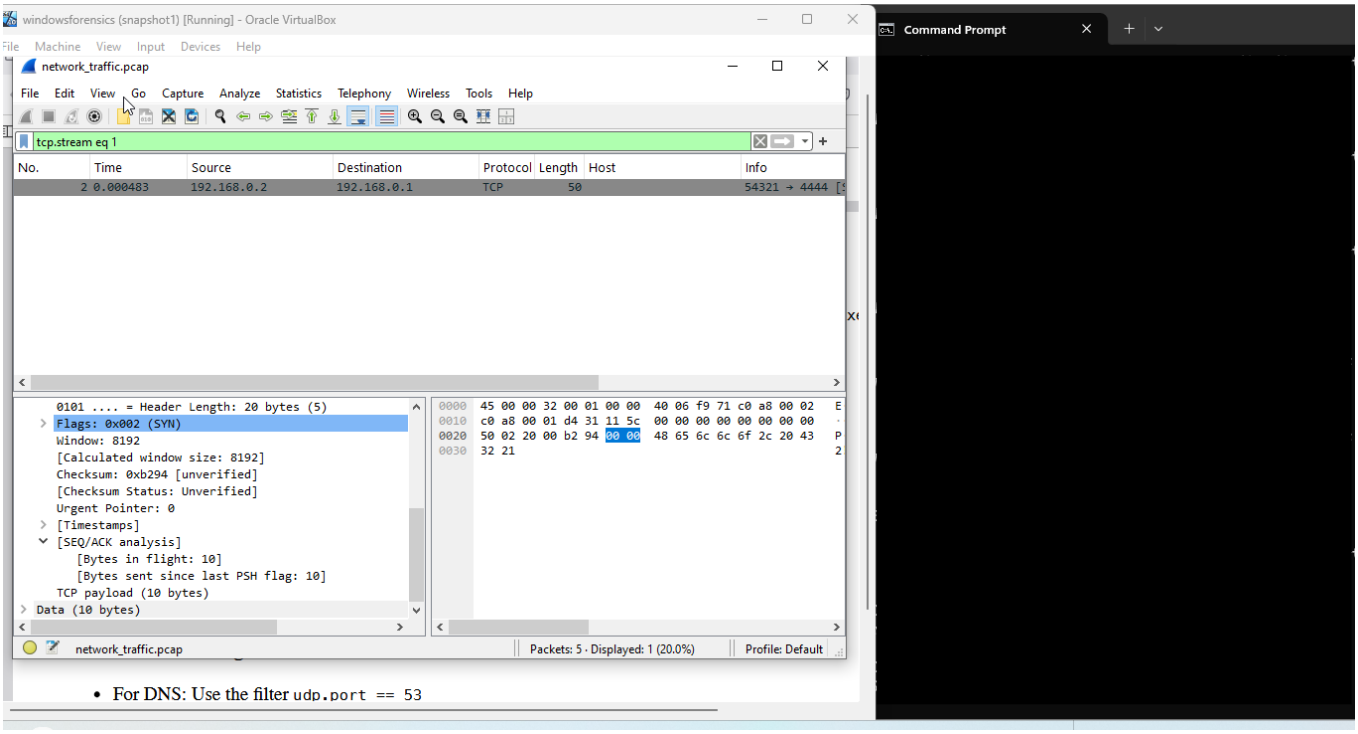


Figure 13:SEQ/ACK,flags inforamtion about command and control

Analyze Data Exfiltration:

Use the filter: tcp.port == 8888

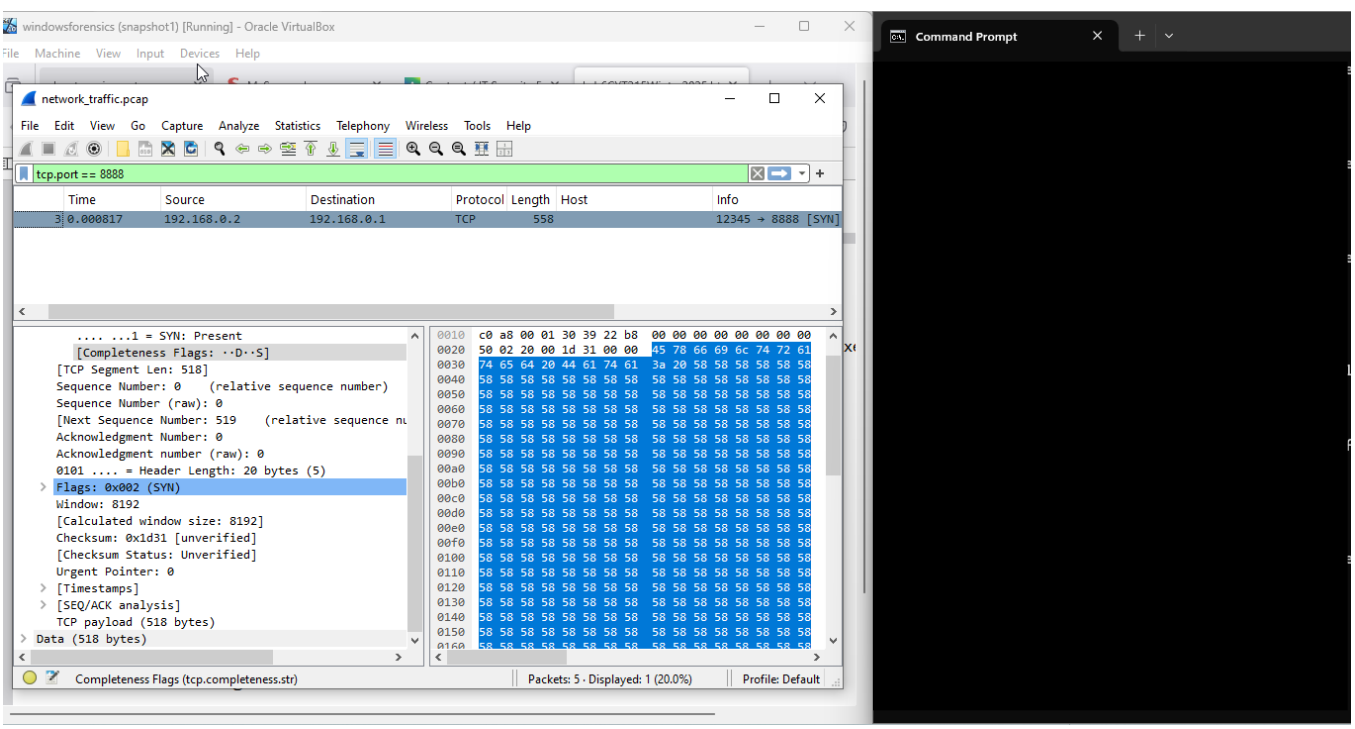


Figure 14:Using tcp.port ==8888 filter

Observe payload size and pattern.

In the length column I can see the size 558. Source IP: 192.168.0.2 ,Destination IP: 192.168.0.1,Protocol: TCP, Source Port: 12345, Destination Port: 8888, Packet Length: 558 bytes,TCP Segment Length: 518 bytes,Flags: 0x002 (SYN) - Indicates that this is a TCP SYN packet (beginning of a connection handshake).

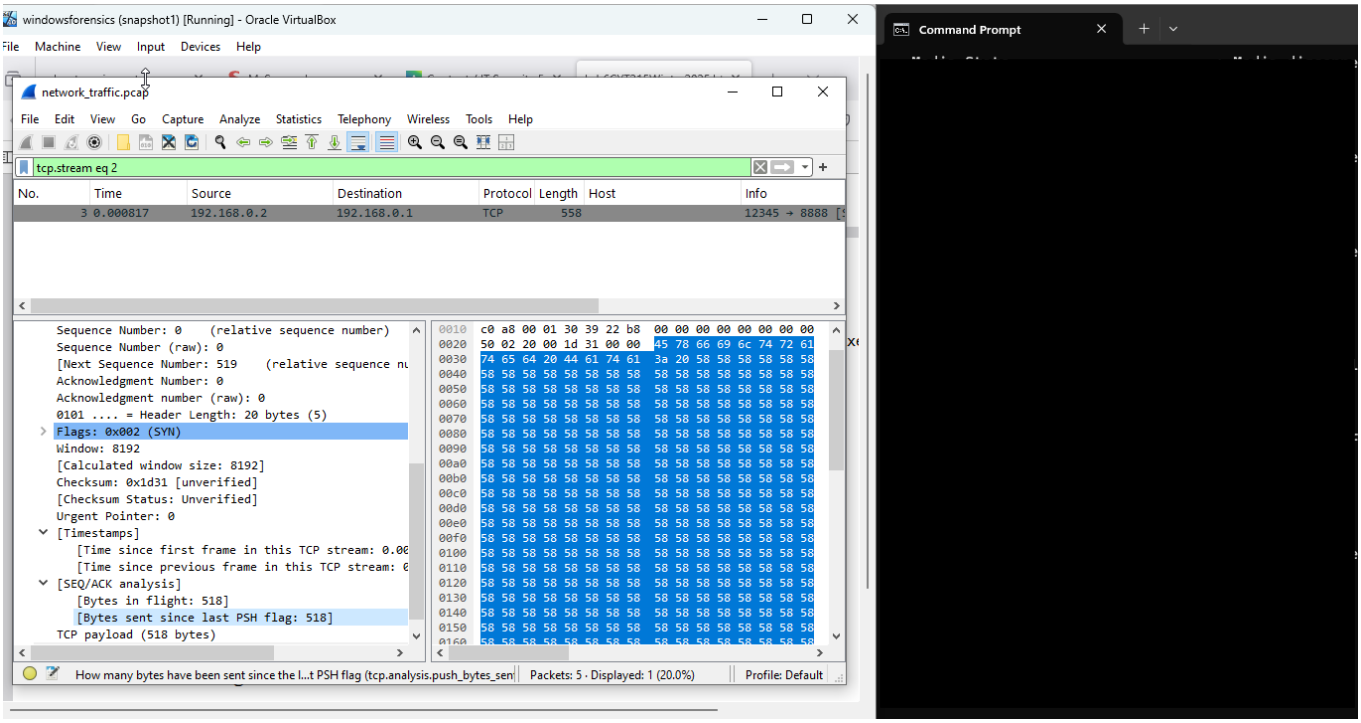


Figure 15:SEQ/ACK analysis

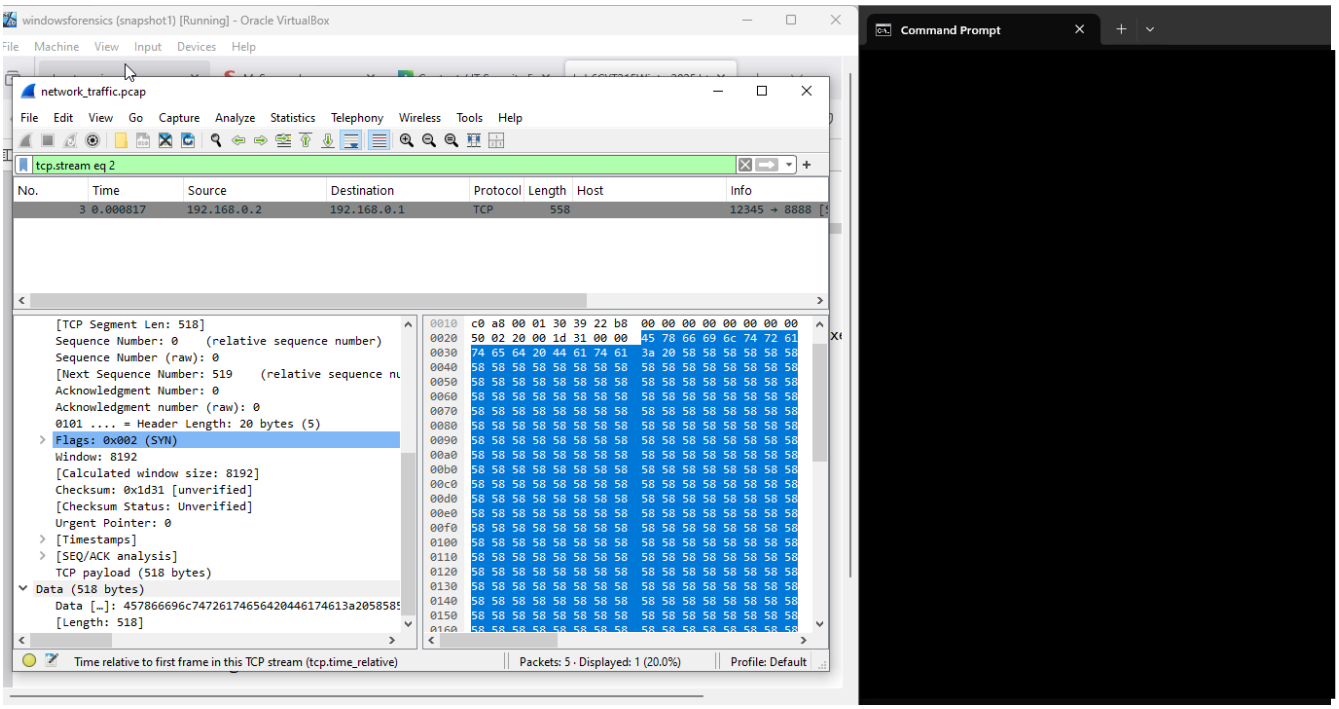
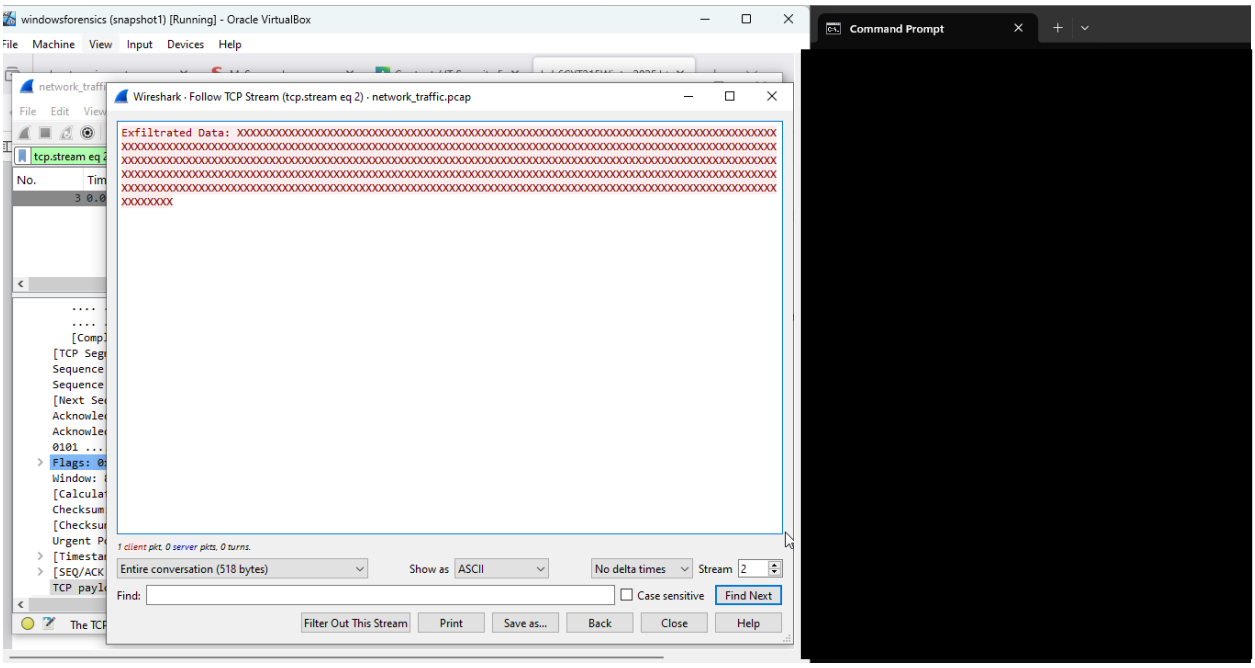


Figure 16:Data 518 bytes

Follow TCP stream to view the exfiltrated data.


windowsforensics (snapshot1) [Running] - Oracle VirtualBox
File Machine View Input Devices Help
Wireshark · Follow TCP Stream (tcp.stream eq 2) · network_traffic.pcap
Exfiltrated Data: XXXX

- Data exfiltration (malicious software using a non-standard port).
- Covert channels for C2 (Command & Control).

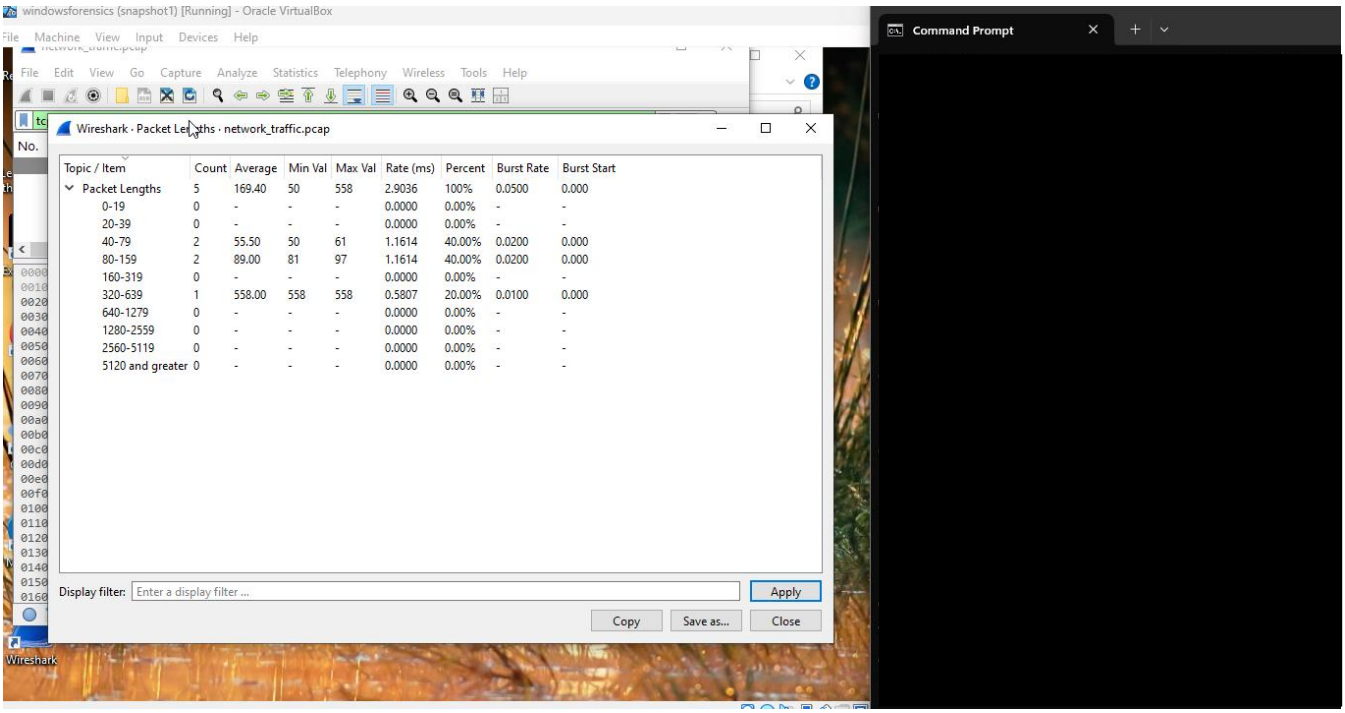


Figure 18:Packet length

Pattern:

- The hex data shows a repeating sequence of 58 58 58 ....
- This corresponds to XXXXX... in ASCII.
- The data seems structured, possibly padding or encoding.
- The repetition might indicate data obfuscation, or a placeholder used in transmission.
- Data exfiltration disguised as normal traffic.
- 1 packet was transferred.

Analyze timestamps to check for unusual transfer times

In Figure 9, In time column I can see 0.000817 . That means this packet was captured 0.000817 seconds (or 817 microseconds) after the first packet in the capture.

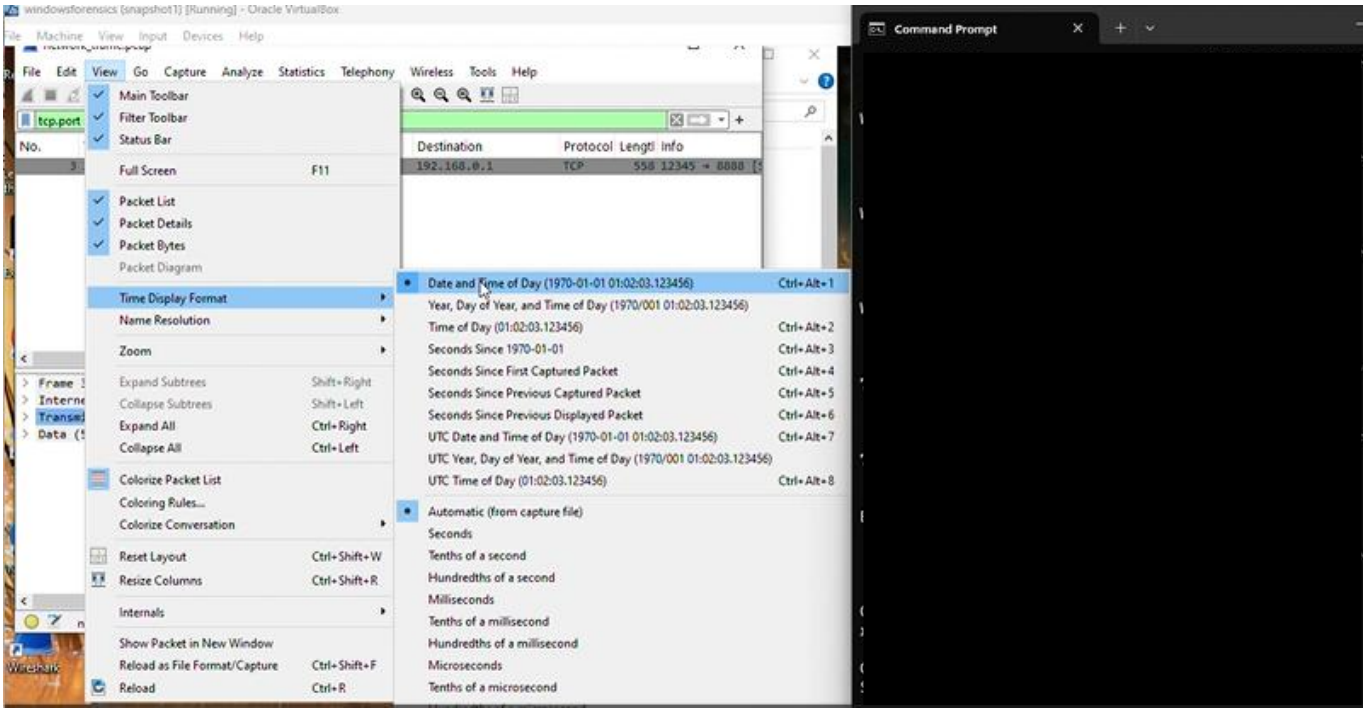


Figure 19:Viewing Time

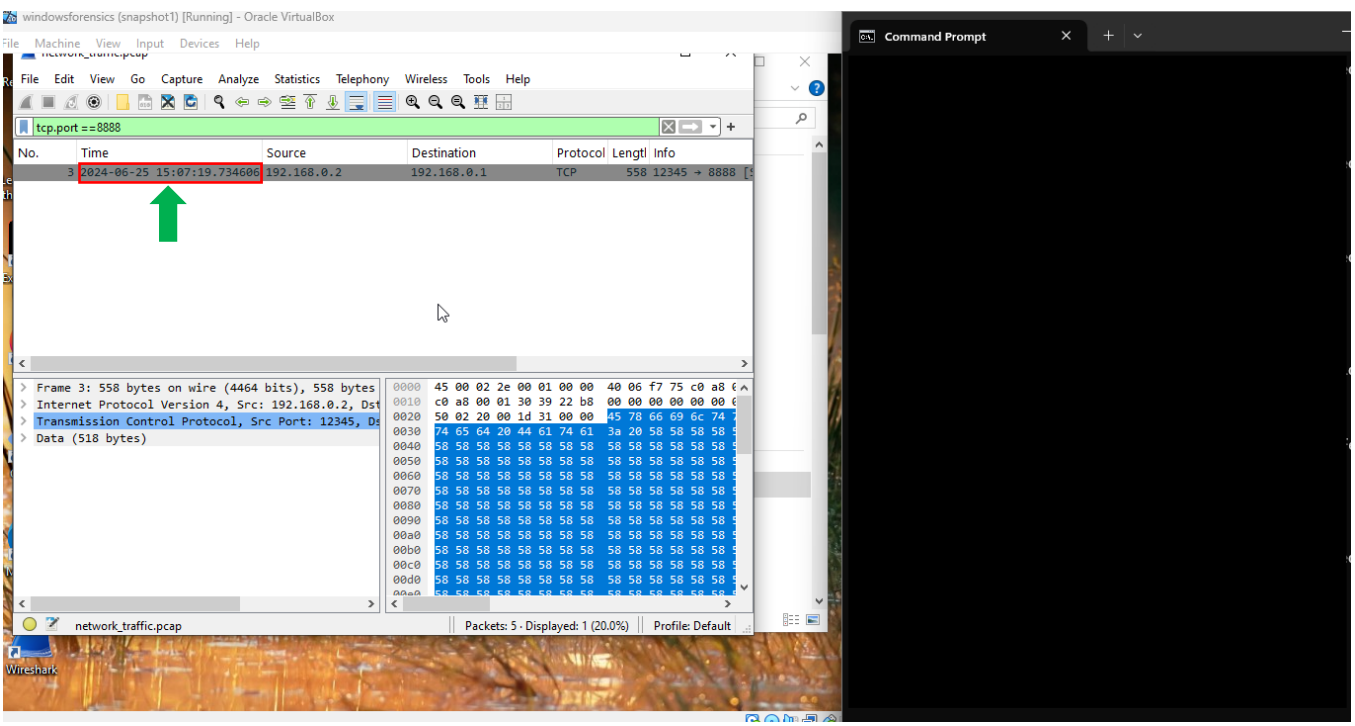


Figure 20:Timestamping

Time is 25th June 2024 3:07 pm.

Differentiate Benign Traffic:

For DNS: Use the filter udp.port == 53

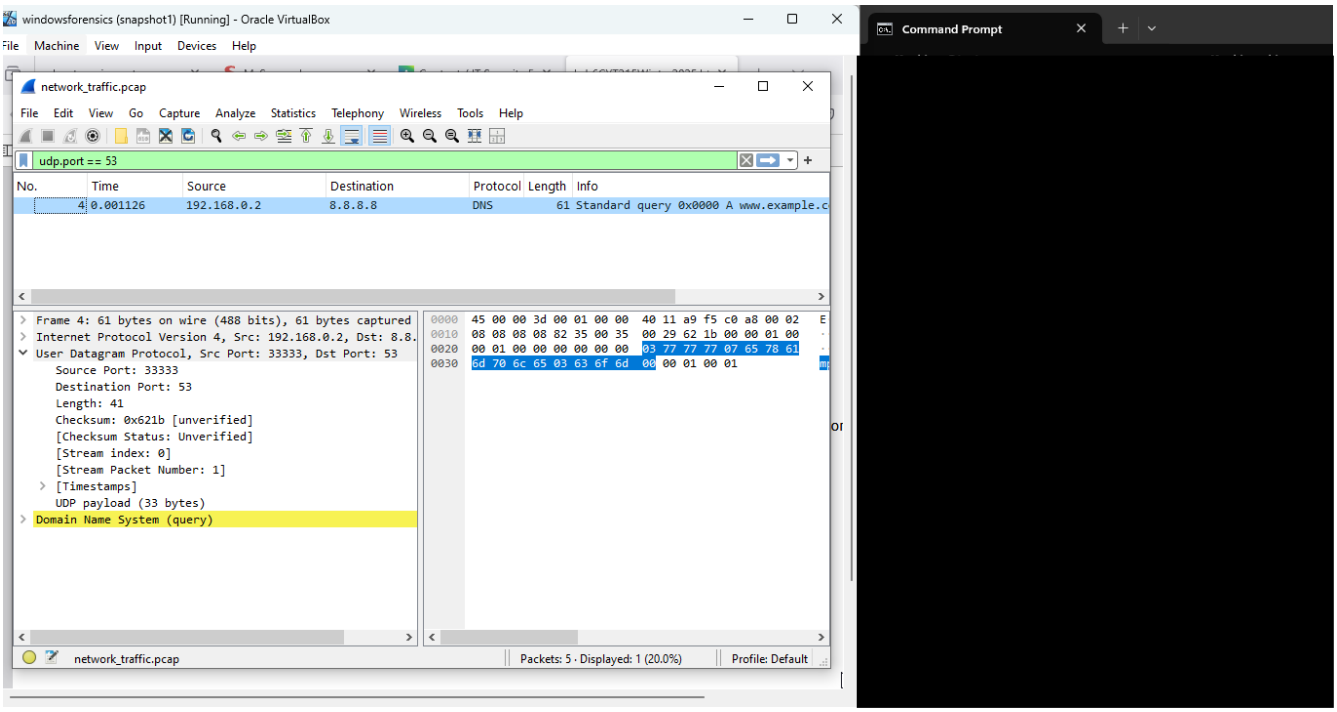


Figure 21loaded filter for DNS

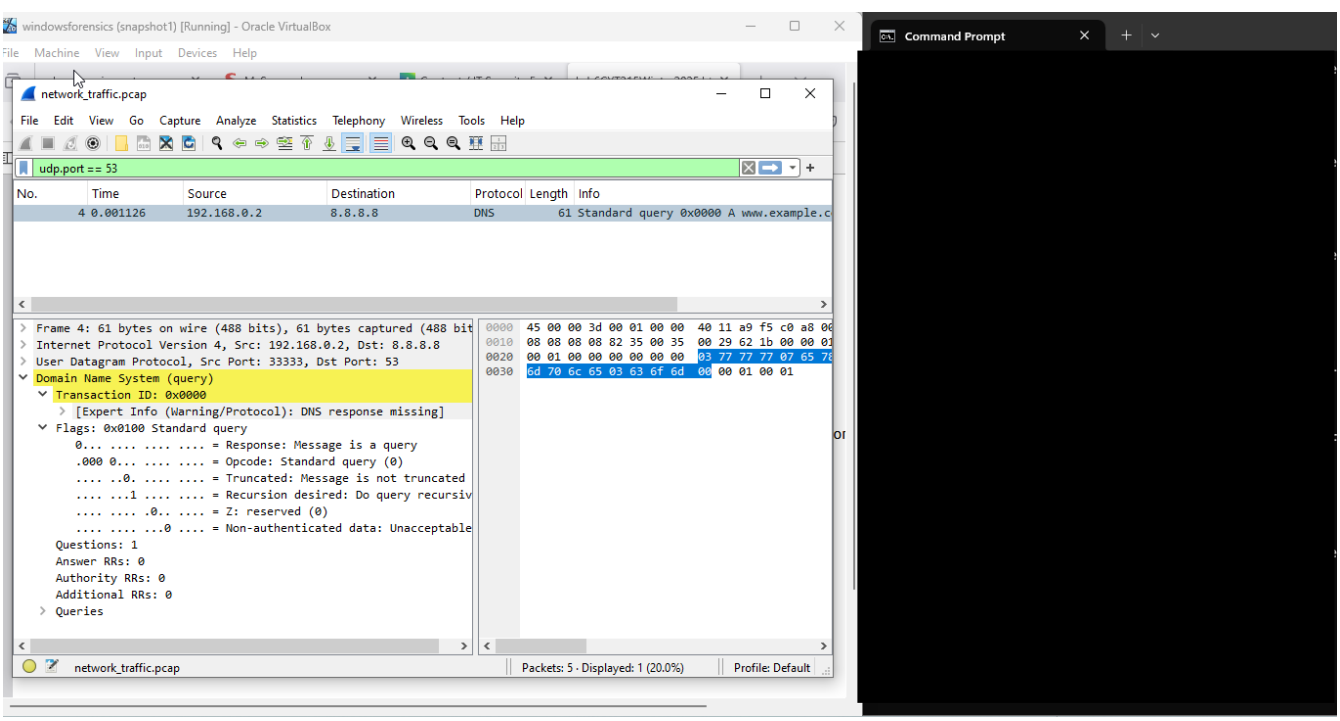


Figure 22: Transaction Information about domain name system(1)

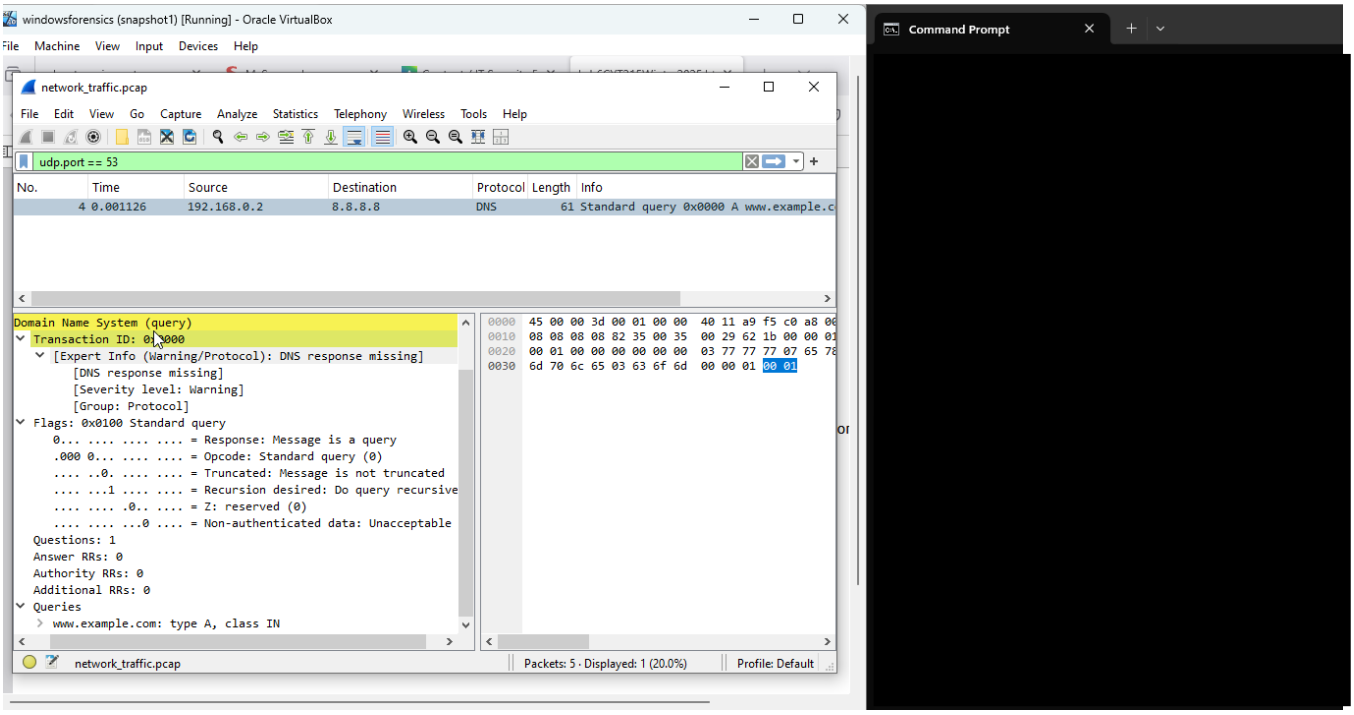


Figure 23:Transaction Information(2)

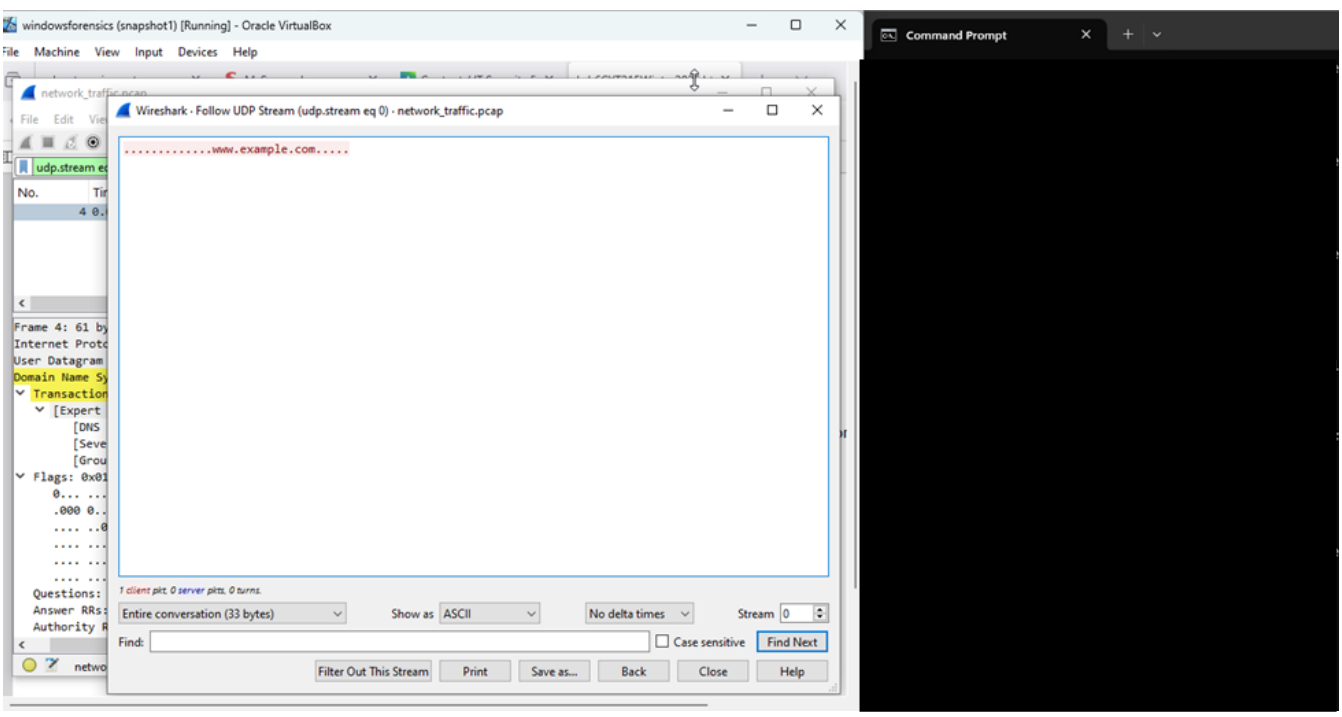


Figure 24:UDP stream

For HTTP: Use the filter http.request.method == "GET" && http.host == www.example.com

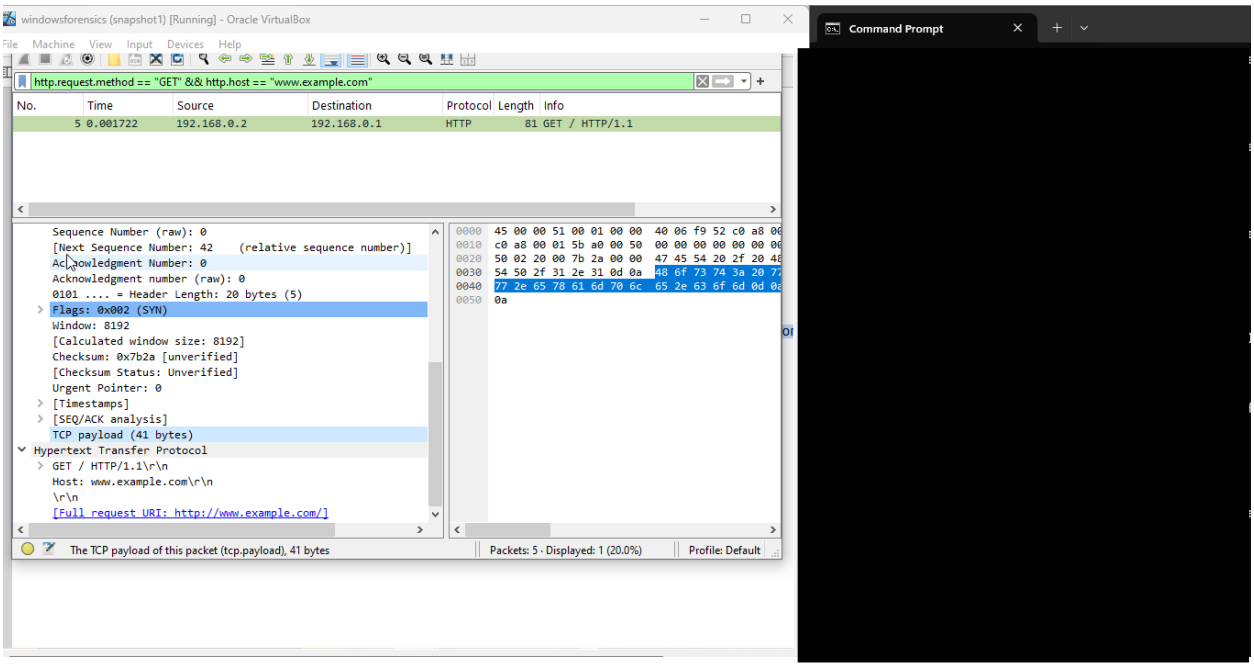


Figure 25:HTTP request filter

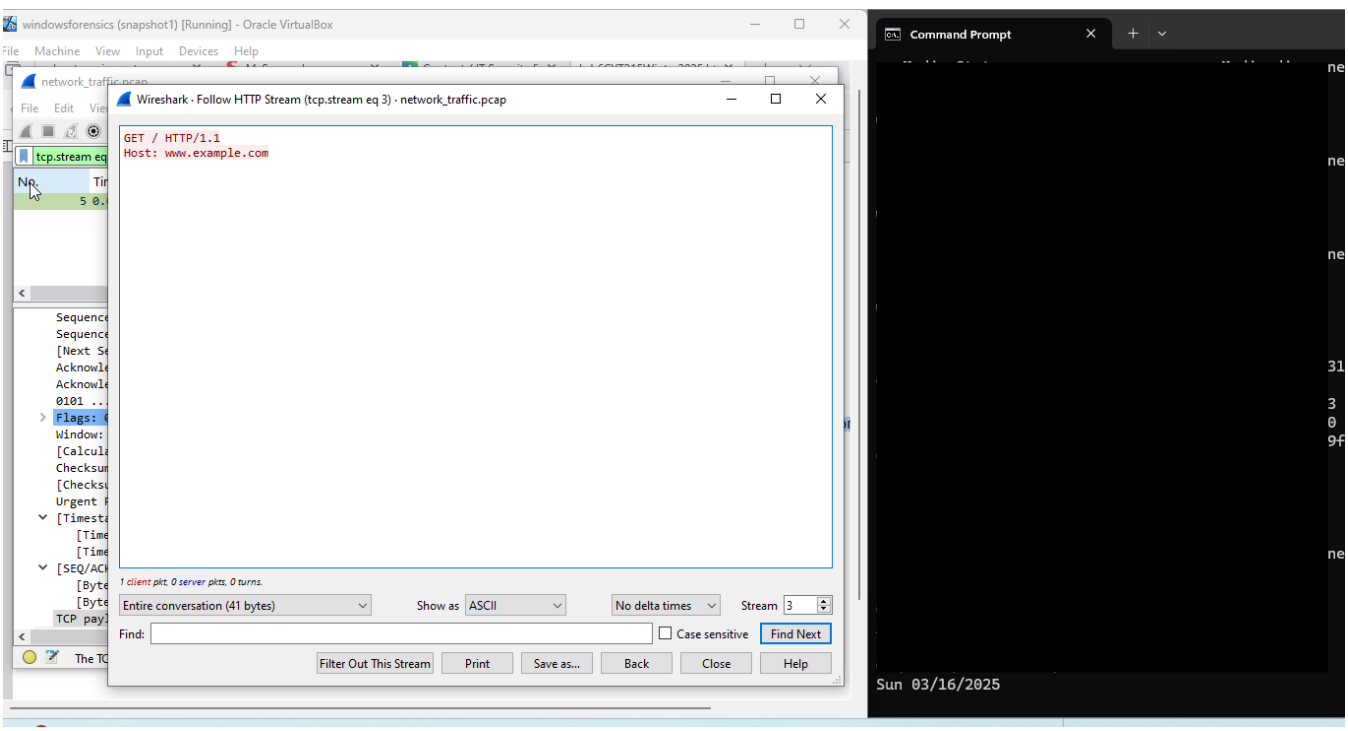


Figure 26:HTTP stream

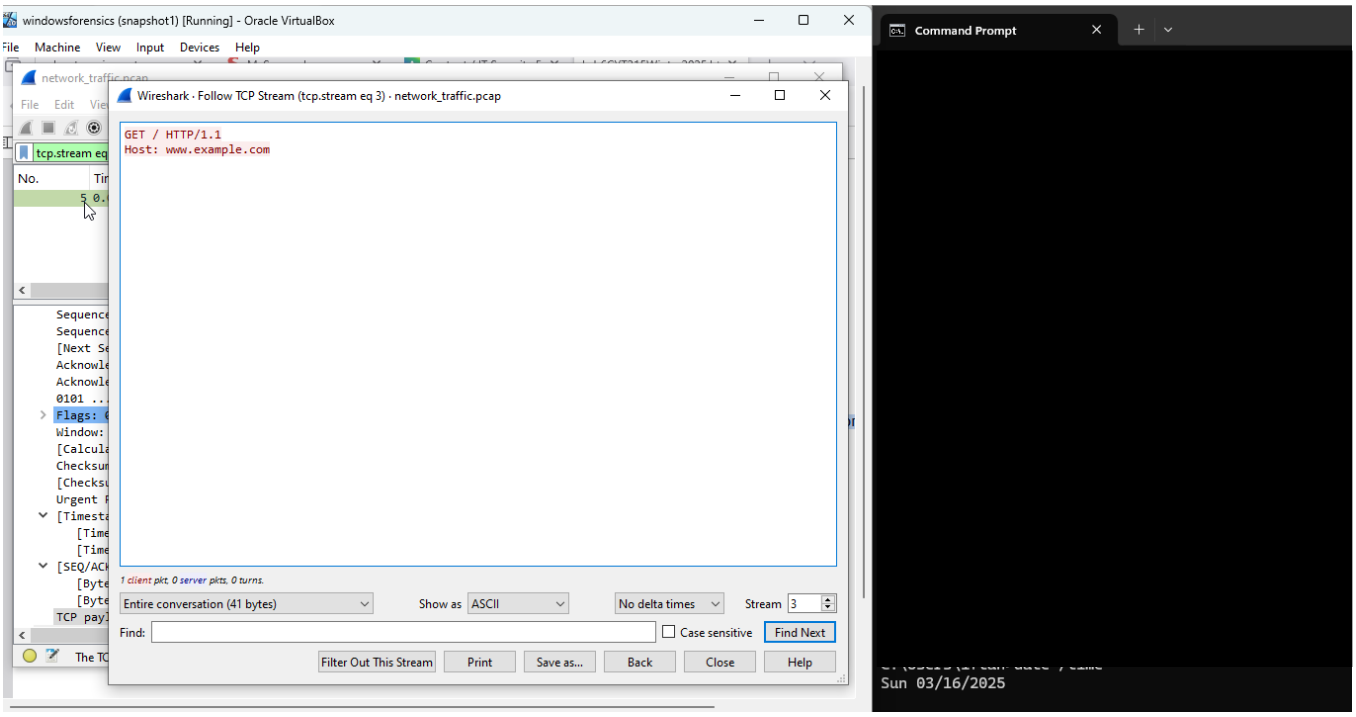


Figure 27:TCP stream

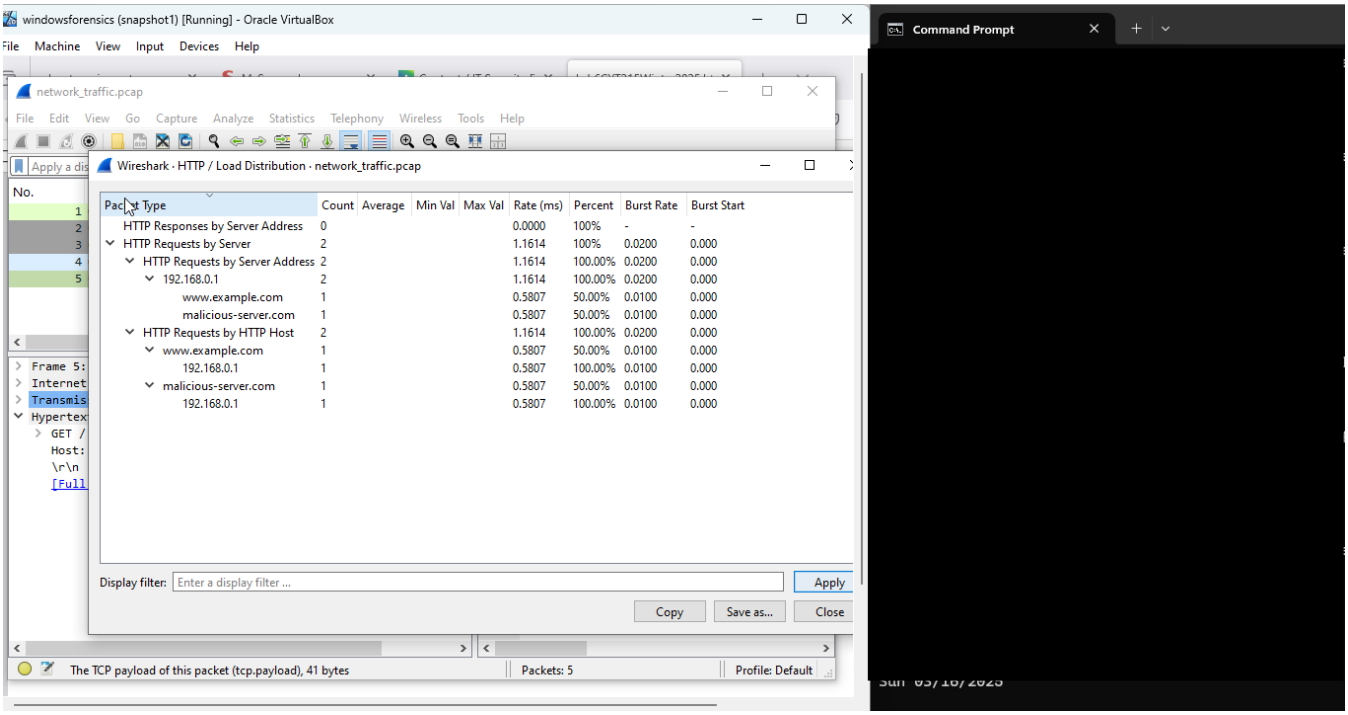


Figure 28:HTTP /Load distribution

Comparison Table: Benign DNS vs. HTTP Traffic

Feature	Benign DNS Traffic	Benign HTTP Traffic
Purpose	Resolving domain names to IP addresses	Requesting web content (pages, images, scripts)
Protocol	UDP (sometimes TCP for large responses)	TCP
Common Ports	53 (UDP/TCP)	80 (HTTP) or 443 (HTTPS)
Source IP	Client's local IP (example-192.168.x.x)	Client's local IP (example-192.168.x.x)
Destination IP	Trusted DNS servers (Google 8.8.8.8, Cloudflare 1.1.1.1, ISP DNS)	Trusted web servers (www.example.com or CDN IPs)
Request Format	Contains domain name in the Query Name field	Contains GET /resource in the request line
Response Format	Returns IP address (e.g., 93.184.216.34 for example.com)	Returns HTML, JSON, images, or other web content
Payload Size	Usually small (64-512 bytes)	Varies based on web content, typically larger
Frequency	Occurs when a user browses, apps connect to servers	Frequent during web browsing
Encryption	DNS is usually plaintext (unless using DoH/DoT)	HTTP is plaintext; HTTPS is encrypted
Legitimate Example	Query: www.google.com -> Response: 142.250.190.78	GET /index.html HTTP/1.1 Host: www.example.com
Indicators of Malicious Traffic	- High volume of DNS requests to random subdomains (DNS tunneling) - Queries for known malware domains	- Unusual or repetitive GET requests to unknown hosts - Data exfiltration in GET parameters